Arch Linux Installation on VMware | Beginner Tutorial

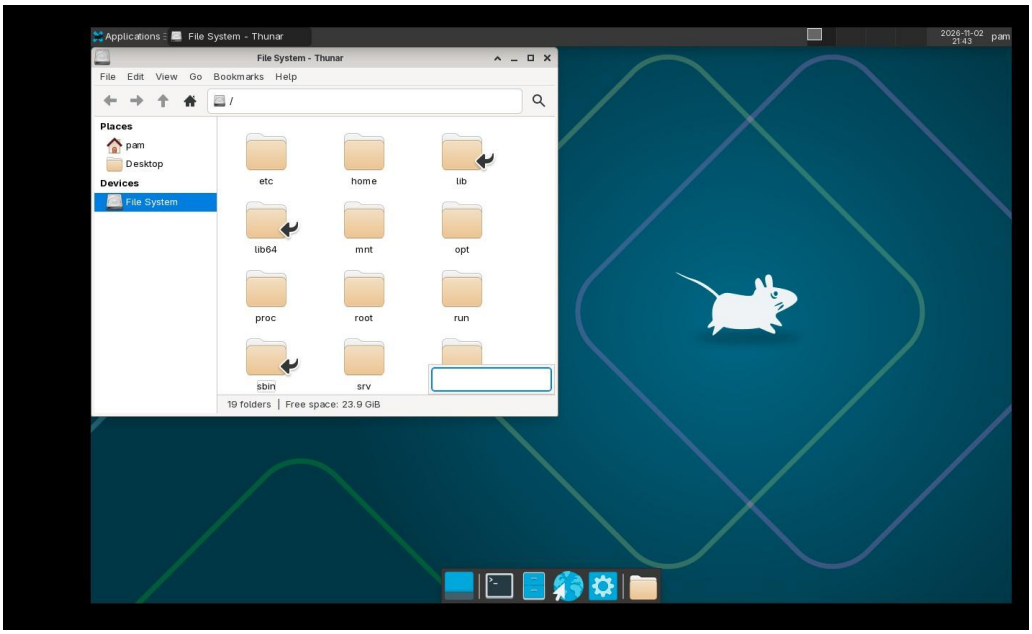
Arch Linux Installation on VMware -Step-by-Step Guide

Written by: Pajok Maluk

System built: `archi` — Arch Linux with Xfce and Zsh
Platform: VMware Workstation Pro 17 on Windows 11
Installation method: Manual installation with the official Arch Linux ISO

The goal is to make each step easy to follow for someone learning Linux: what to click, what to type, why the command is needed, and how to confirm the result. Screenshots below are from my own installation.

By the end, the VM boots into the **Xfce graphical desktop**, supports **SSH**, has **two sudo-enabled users**, uses a **colored Zsh terminal**, runs **Firefox**, and includes an **AUR package built from source**.



My completed Arch Linux Xfce desktop, with the Thunar file manager open.

Table of Contents

1. Prerequisites
2. Create the Virtual Machine
3. Install Arch Linux Manually
4. Configure the System
5. Install and Use the Xfce Desktop
6. Customize Zsh and Install Firefox
7. Build a Package from the AUR
8. Verify the Finished VM

9. References

1. Prerequisites

To follow the same process, prepare:

- A Windows computer with **VMware Workstation Pro** installed.
- The official **Arch Linux x86_64 ISO** from archlinux.org/download.
- Internet access inside the virtual machine.
- At least **4 GB of RAM** and **30 GB of disk space** available to assign to the VM.
- Basic familiarity with copying Linux commands into a terminal.

Read before partitioning: This guide uses an **empty, dedicated VMware virtual disk** named `/dev/sda`. The partitioning and formatting commands erase data on that disk. Verify your device name with `lsblk` before running them. Do not use these commands against your Windows drive or a disk containing files you want to keep.

My VM configuration

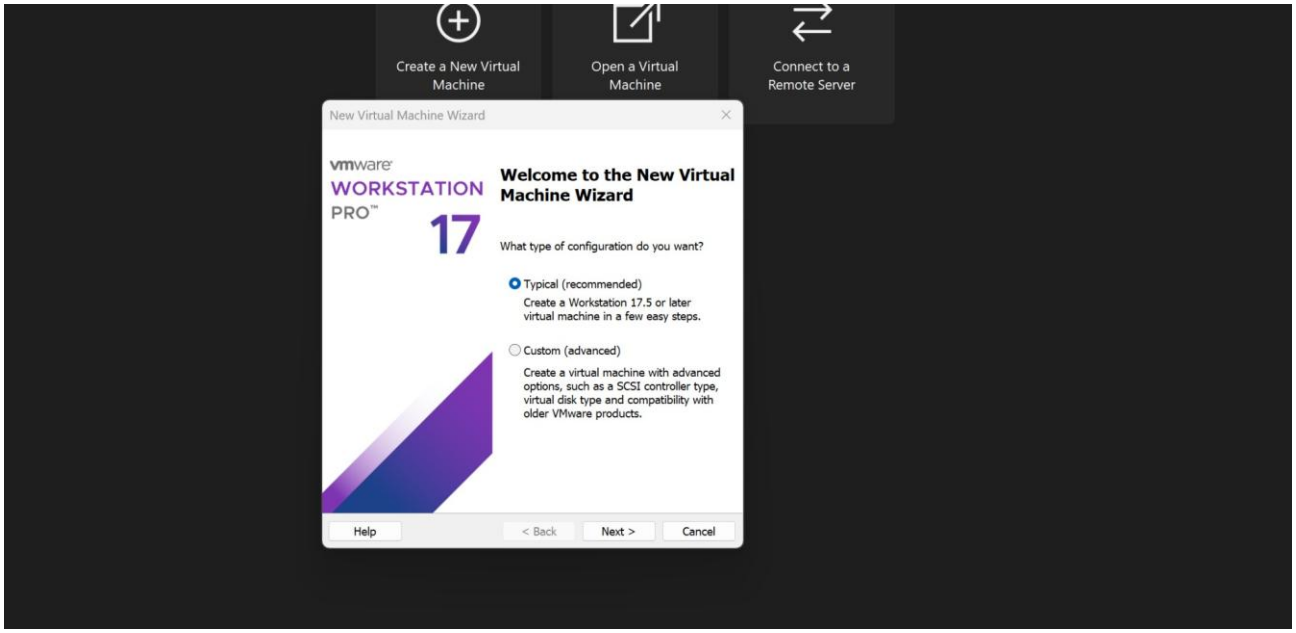
Setting	Value
VM name / hostname	`archlinux-pam`
Guest operating system	64-bit Linux (x86_64)
Firmware	UEFI
Processors	2 virtual CPUs
Memory	4096 MB (4 GB)
Virtual disk	30 GB (SCSI)
Networking	NAT
Main username	`pam`
Second username	`wujek`

2. Create the Virtual Machine

Step 1 — Start the VMware setup wizard

1. Open **VMware Workstation Pro**.
2. Choose **Create a New Virtual Machine**.
3. Select the Linux guest operating system with 64-bit architecture.

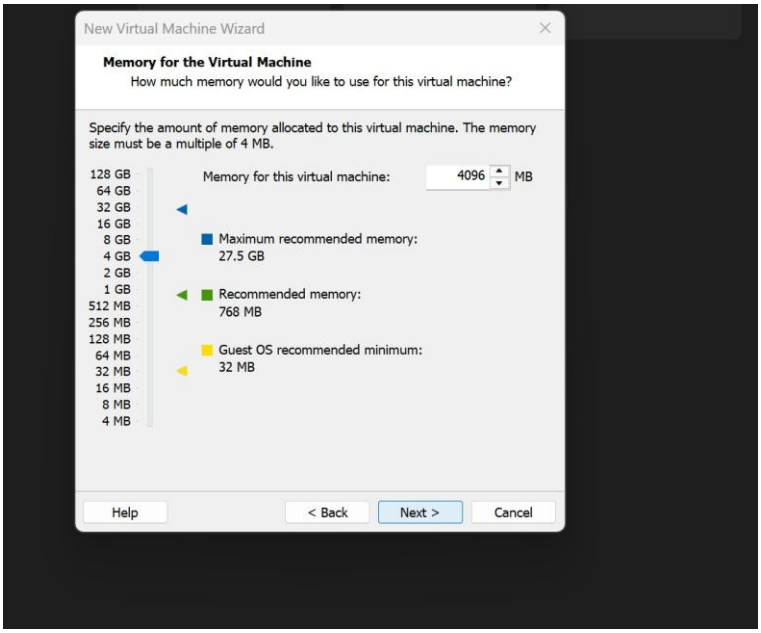
4. Name the VM `archi-pam`.



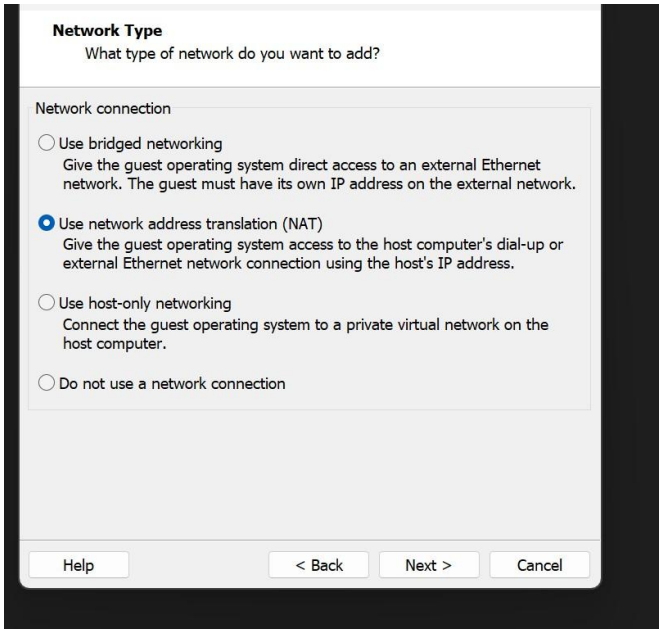
The VMware Workstation new virtual machine wizard.

Step 2 — Allocate the VM hardware

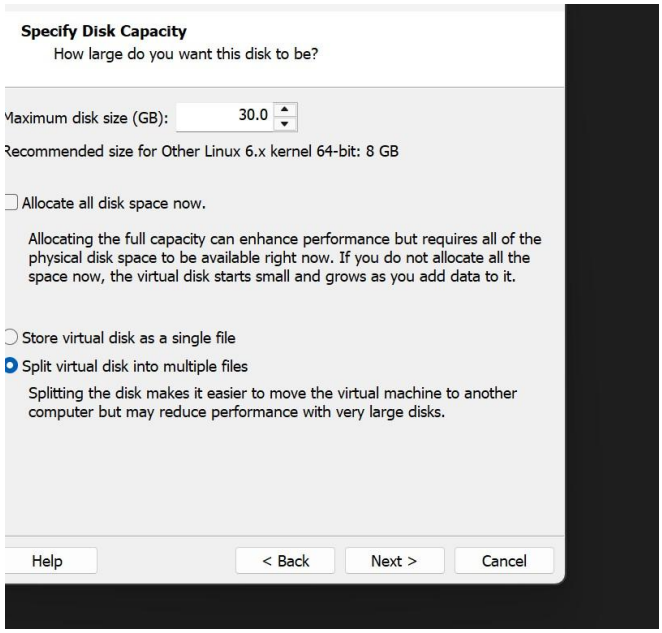
1. Assign **4096 MB RAM**.
2. Set **2 virtual processors**.
3. Create a **30 GB virtual disk**.
4. Use a **NAT network adapter** so the VM can access the internet through the host computer.
5. Use **UEFI** firmware for the installation.



Allocating 4 GB of RAM.



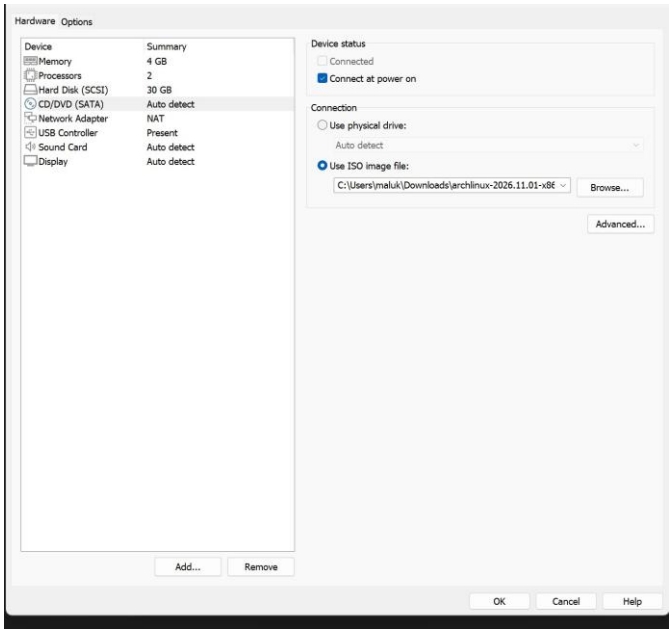
Selecting NAT networking.



Creating a 30 GB virtual disk.

Step 3 — Connect the Arch Linux ISO

Open **VM Settings → CD/DVD**, choose the downloaded Arch Linux ISO, and ensure it is connected at power on. Start the VM and enter the Arch live terminal. You should see a prompt similar to `root@archiso`.



The Arch ISO attached to the VM's virtual CD/DVD drive.

---

3. Install Arch Linux Manually

The following commands are run in the **Arch ISO live terminal as root** until the `arch-chroot` step.

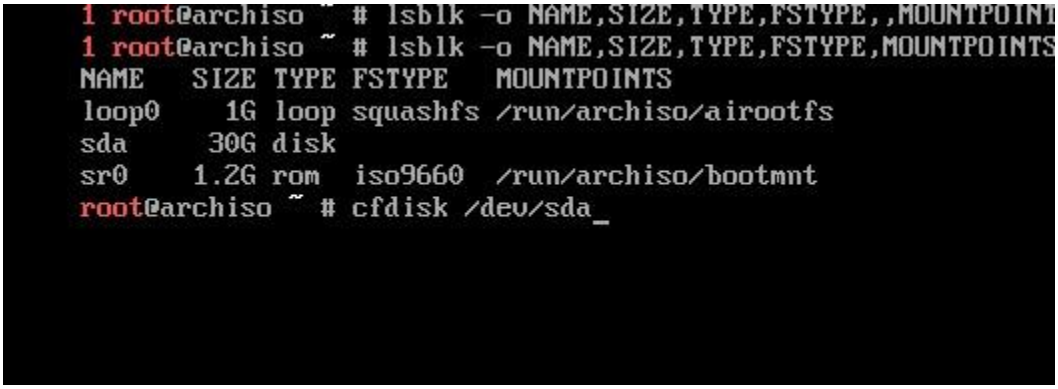
Step 4 — Check UEFI, networking, and time

```
ls /sys/firmware/efi/efivars                # Confirm that the VM booted using UEFI ping
-c 4 archlinux.org                      # Send four packets to test internet access
timedatectl status                          # Check that the system clock is synchronized
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS   # Identify the empty virtual disk
```

The `ping` command should return replies. My installation disk appeared as **`/dev/sda`**, with **30 GiB** of space. The Arch ISO is a separate, read-only device.



Testing the network before installation.



Using `lsblk` to locate the empty 30 GB disk.

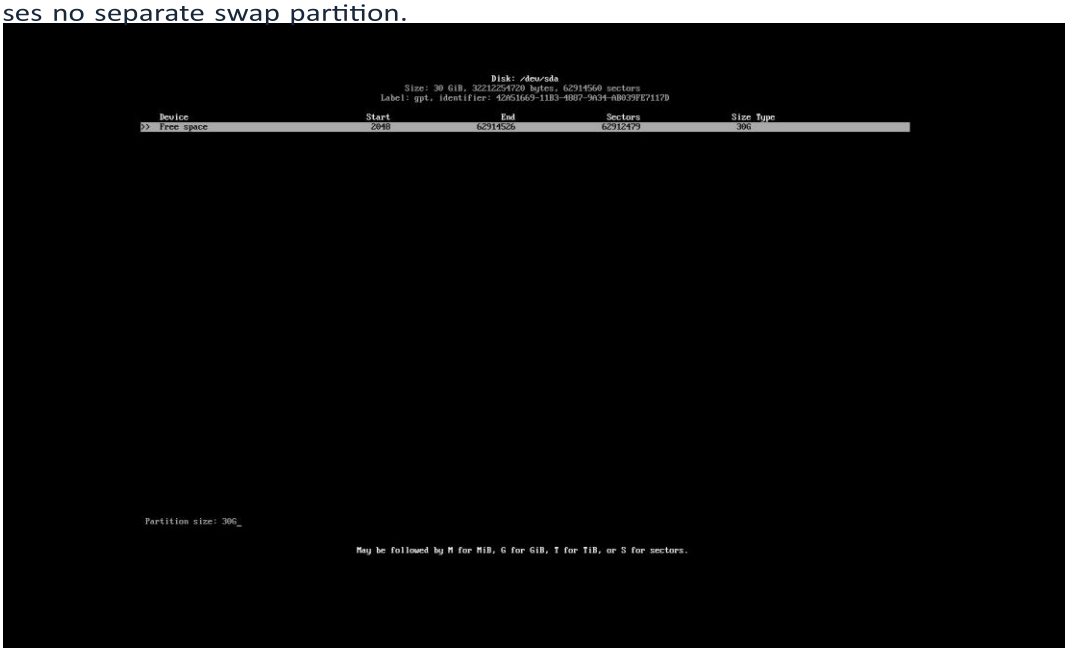
Step 5 — Partition the disk with cfdisk

```
cfdisk /dev/sda   # Open the partition editor for the empty VMware disk
```

Select **GPT** as the partition label type, then create the partitions below:

Partition	Size	Type	Purpose
`/dev/sda1`	1 GiB	EFI System	Files needed to boot in UEFI mode
`/dev/sda2`	Remaining ~29 GiB	Linux filesystem	Root filesystem `/`

In `cfdisk`, select **New** to create each partition, choose the correct **Type**, and finish with **Write → yes → Quit**. This setup uses no separate swap partition.



Creating a 1 GiB EFI System Partition.

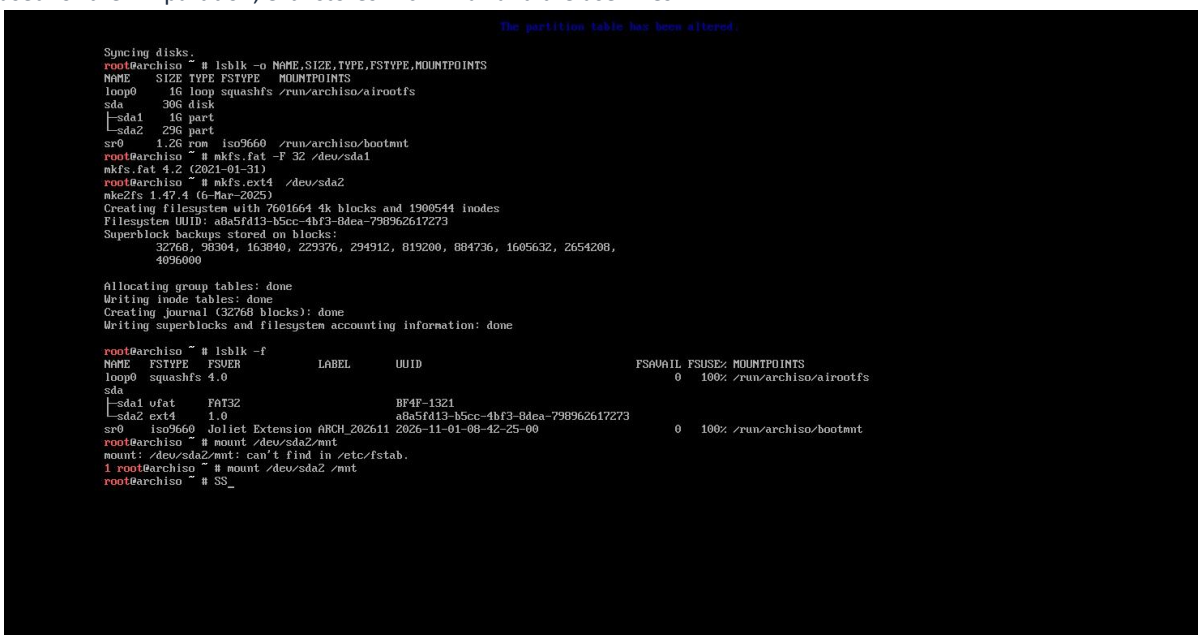


The completed GPT layout with EFI and Linux partitions.

Step 6 — Format the new partitions

```
mkfs.fat -F 32 /dev/sda1  # Create a FAT32 filesystem for EFI
mkfs.ext4 /dev/sda2       # Create an ext4 filesystem for the main system
lsblk -f                  # Verify both filesystem types
```

FAT32 is used for the EFI partition; ext4 stores Arch Linux and the user files.

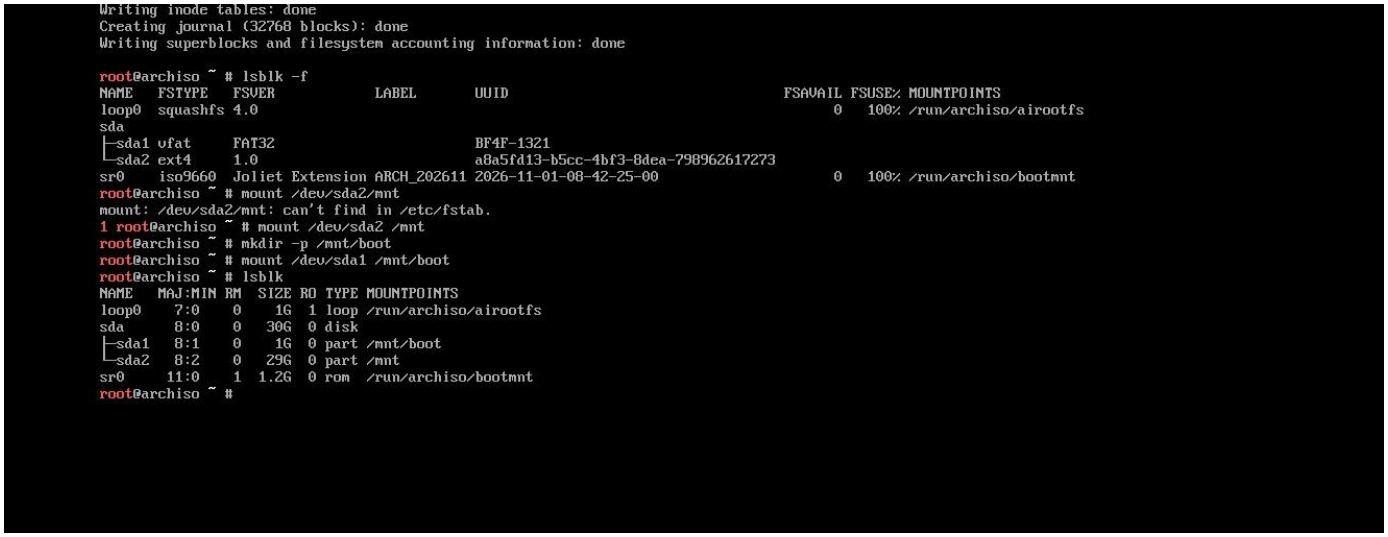


Formatting the EFI and root partitions.

Step 7 — Mount the filesystems

```
mount /dev/sda2 /mnt        # Mount the root filesystem for installation
mkdir -p /mnt/boot          # Create the EFI mount directory
mount /dev/sda1 /mnt/boot   # Mount the EFI partition at /boot
lsblk                       # Confirm both mount points
```

You should see `/dev/sda2` mounted at `/mnt` and `/dev/sda1` at `/mnt/boot`.



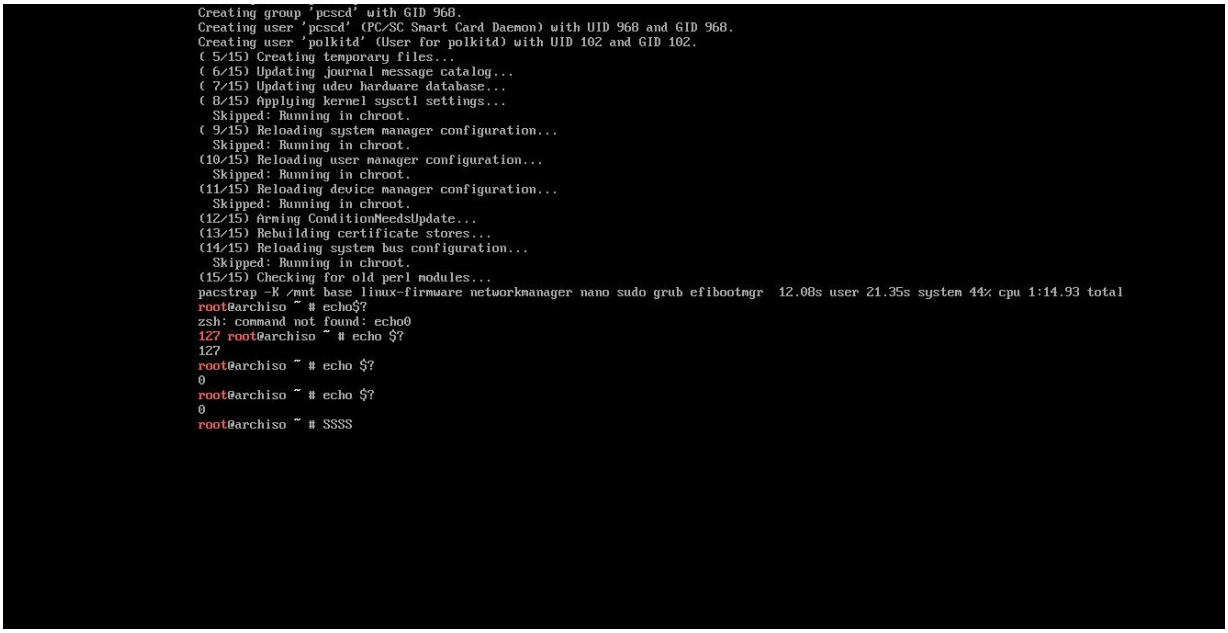
Verifying the two mount points.

Step 8 — Install Arch Linux with pacstrap

`pacstrap` copies the base operating system and selected packages to the mounted disk. The command below combines the packages used for this finished setup into one reproducible install command:

```
pacstrap -K /mnt base linux linux-firmware networkmanager nano sudo grub efibootmgr openssh zsh git
```

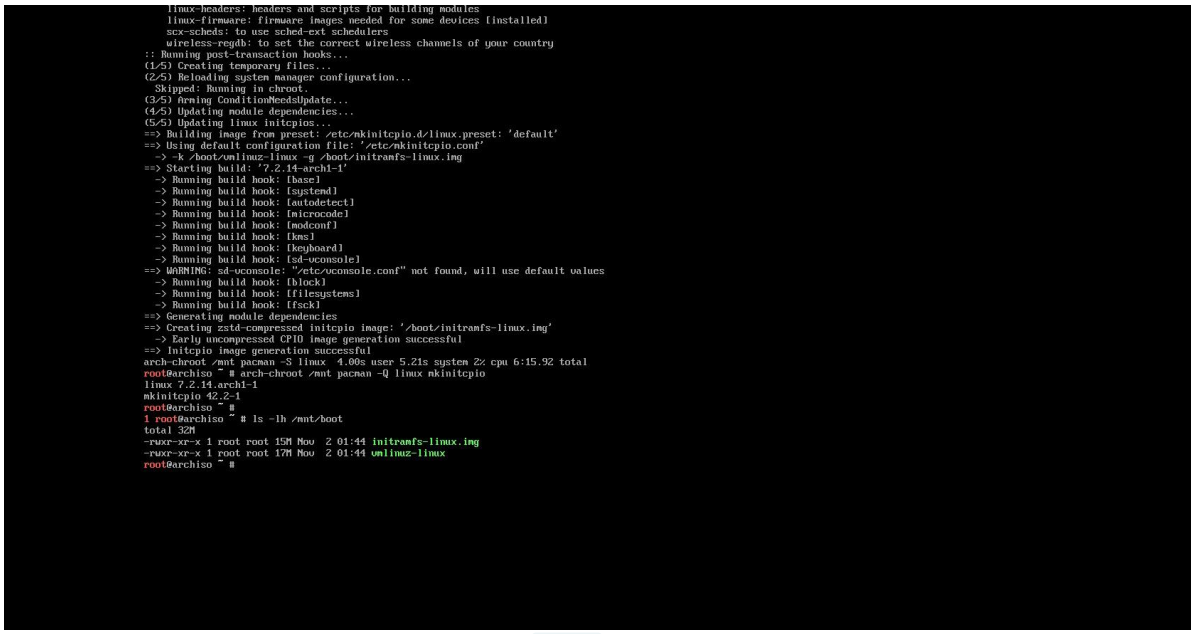
The key packages include the `linux` kernel, `linux-firmware`, networking tools, Zsh, SSH, and GRUB. Let the package installation complete before moving on.



Installing the Arch base system with `pacstrap`.

Check the boot images:

```
ls -lh /mnt/boot  # Look for vmlinuz-linux and initramfs-linux.img
```

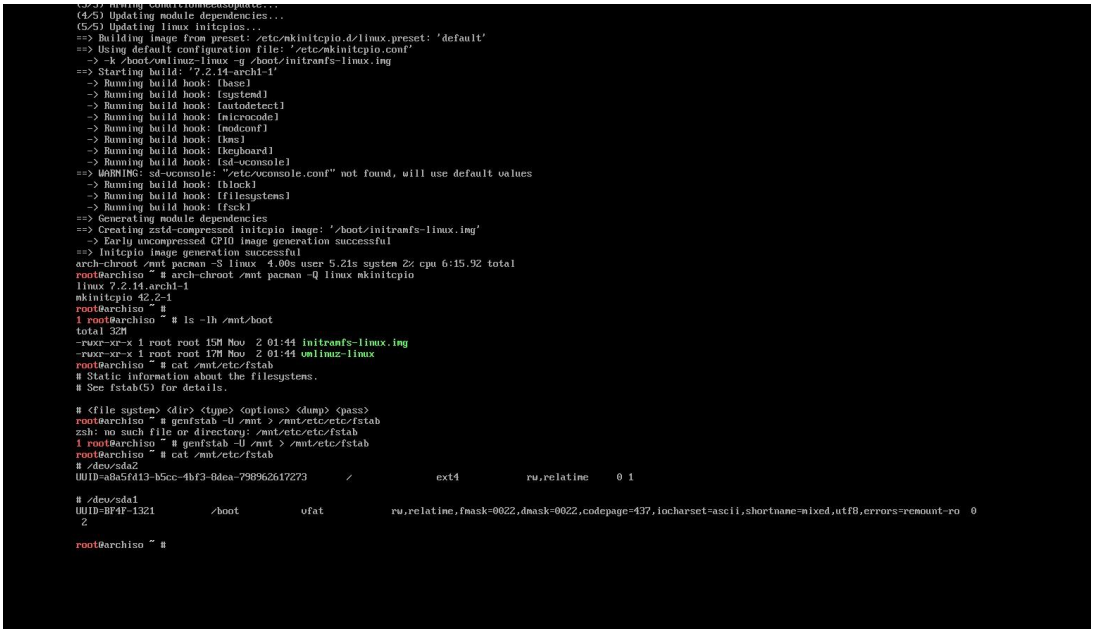


The Linux kernel and initramfs are present under `/boot`.

Step 9 — Generate the filesystem table

```
genfstab -U /mnt > /mnt/etc/fstab   # Record mounted partitions using UUIDs
cat /mnt/etc/fstab                  # Read the generated table
```

This file tells Arch which partitions to mount automatically at startup.



Checking `/etc/fstab` entries for `/` and `/boot`.

Step 10 — Enter the installed environment

```
arch-chroot /mnt  # Work inside the installed Arch system rather than the live ISO
```

From this point until `exit`, run the commands **inside the chroot as root**

---

4. Configure the System

Step 11 — Set the time zone and system clock

```
ln -sf /usr/share/zoneinfo/America/Chicago /etc/localtime   # Use Central Time
hwclock --systohc                                           # Set the hardware clock
```

Replace `America/Chicago` with your own time zone if you live elsewhere.

Step 12 — Set the system locale

Open the locale list:

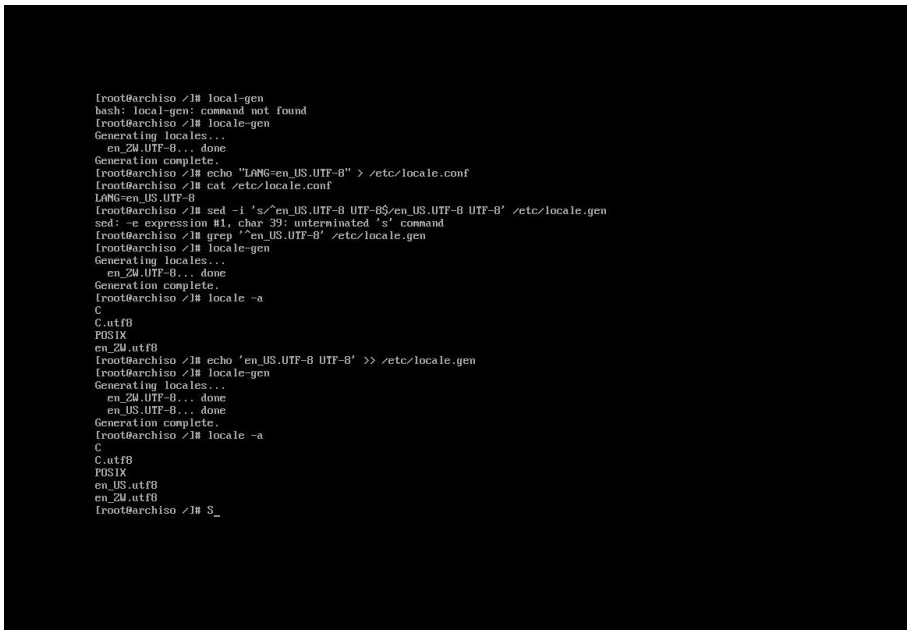
```
nano /etc/locale.gen  # Edit supported language and character-encoding settings
```

Find the line `#en_US.UTF-8 UTF-8` and remove the leading `#` so it reads:

```
en_US.UTF-8 UTF-8
```

In Nano, save with **Ctrl + O**, press **Enter**, then exit with **Ctrl + X**. Run:

```
locale-gen                                  # Generate the selected locale
echo 'LANG=en_US.UTF-8' > /etc/locale.conf  # Make English UTF-8 the default
locale -a                                   # Verify en_US.utf8 is available
```



Generating and verifying the US English locale.

Step 13 — Name the computer

```
echo 'archi-pam' > /etc/hostname   # Set the VM's hostname
nano /etc/hosts                    # Add local name-resolution entries
```

Use the following local host entries in `/etc/hosts`:

```
127.0.0.1   localhost
::1         localhost
127.0.1.1   archi-pam.localdomain archi-pam
```

The hostname appears in the shell prompt after startup. You may choose a different hostname, but use it consistently in both files.

Step 14 — Enable the network service

```
systemctl enable NetworkManager       # Arrange for networking to start during boot
systemctl is-enabled NetworkManager   # Verify that the result is enabled
```



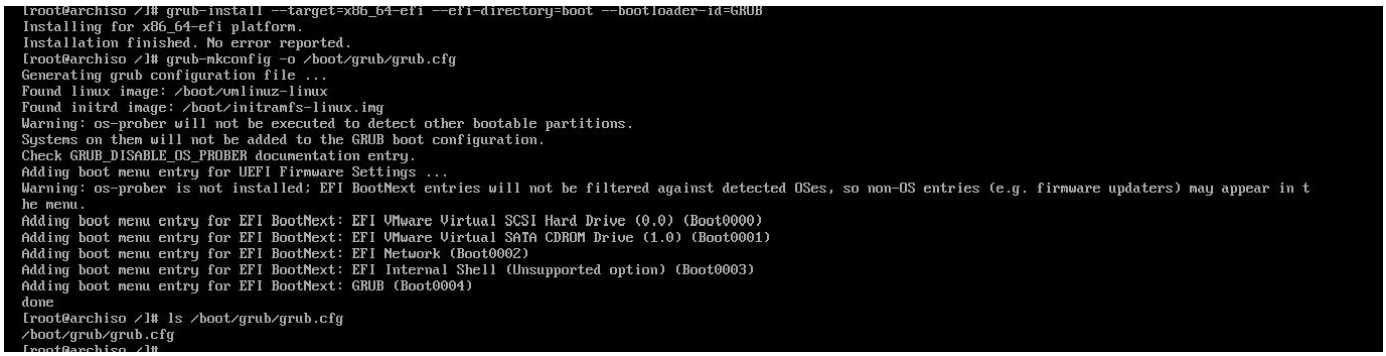
NetworkManager configured to start at boot.

Step 15 — Install the UEFI bootloader

Because the EFI partition was mounted at

```
findmnt /boot  # Confirm /boot is the EFI partition
grub-install --target=x86_64-efi --efi-directory=/boot --bootloader-id=GRUB  # Install the bootloader
grub-mkconfig -o /boot/grub/grub.cfg  # Generate the boot menu
ls /boot/grub/grub.cfg               # Confirm the configuration exists
```

The generated GRUB menu should detect `vmlinuz-linux` and `initramfs-linux.img`.



`/boot`, use `/boot` as the EFI directory:

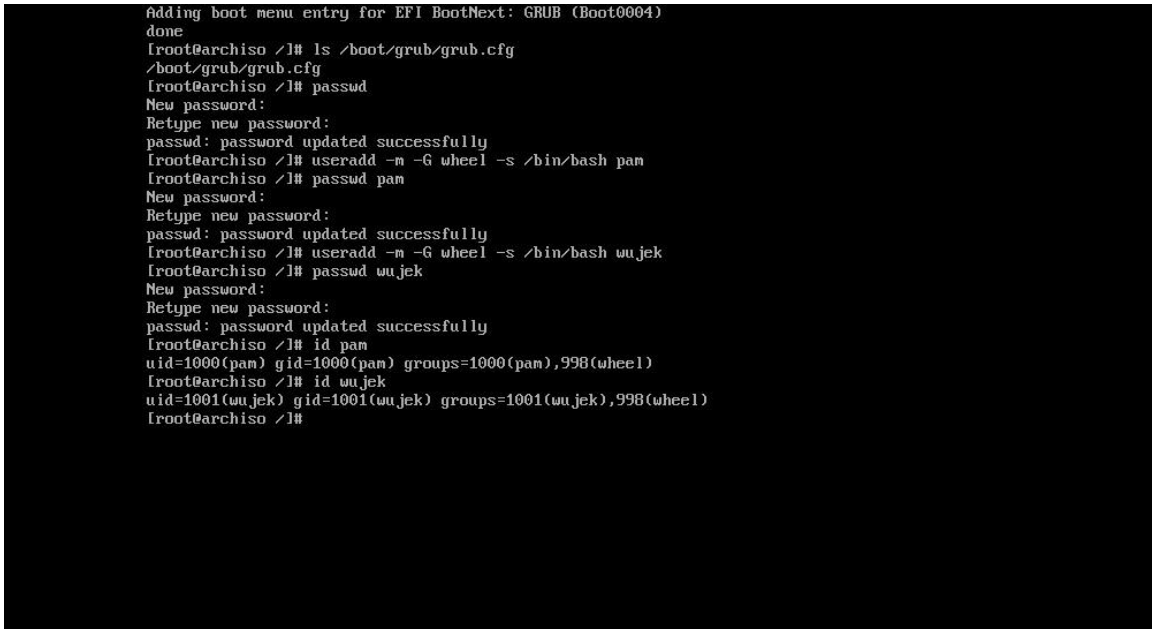
Verification of the installed GRUB configuration and Linux boot image.

Step 16 — Create two user accounts

The `root` account administers the machine. It is better to use a regular account for everyday work. I created `pam` and `wujek`, placing both in the `wheel` group:

```
passwd                                  # Set a password for root
useradd -m -G wheel -s /bin/bash pam   # Create pam with a home directory
passwd pam                              # Set the pam password useradd -m -
G wheel -s /bin/bash wujek # Create a second user passwd wujek
# Set the second user's password id pam                                    #
Verify pam's group membership id wujek                                  #
Verify wujek's group membership
```

Choose your own account names if following this tutorial on a different VM. Password characters will not appear while typing.



Both accounts created with `wheel` membership.

Step 17 — Allow wheel-group users to use sudo

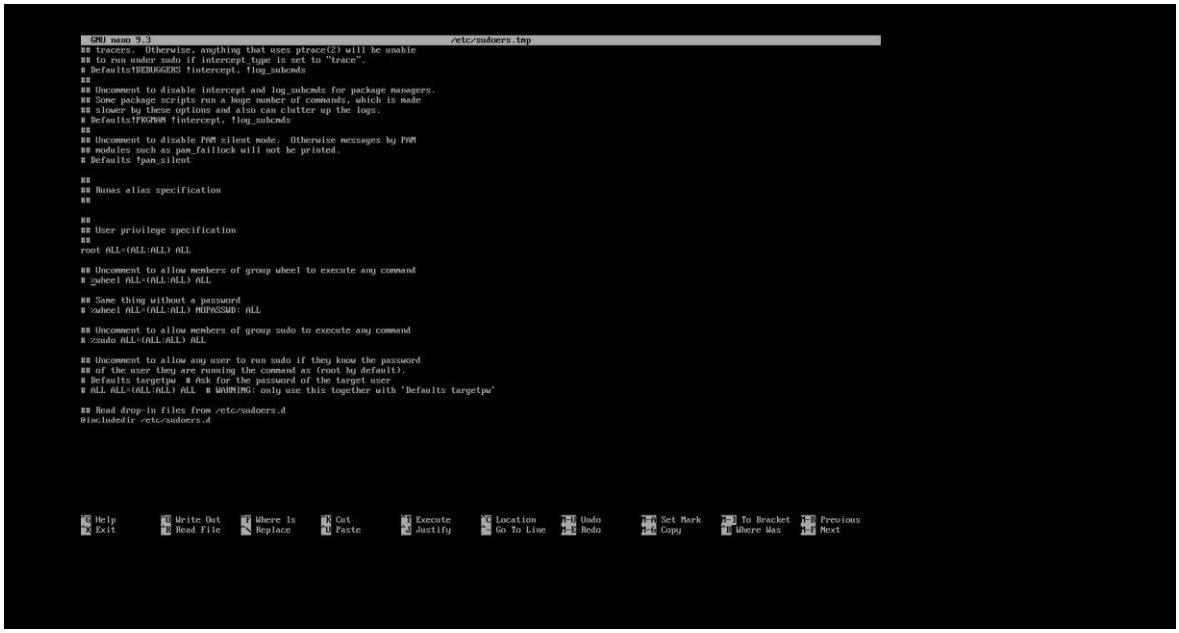
```
EDITOR=nano visudo  # Open the sudoers file with syntax checking
```

Find the wheel rule and uncomment it:

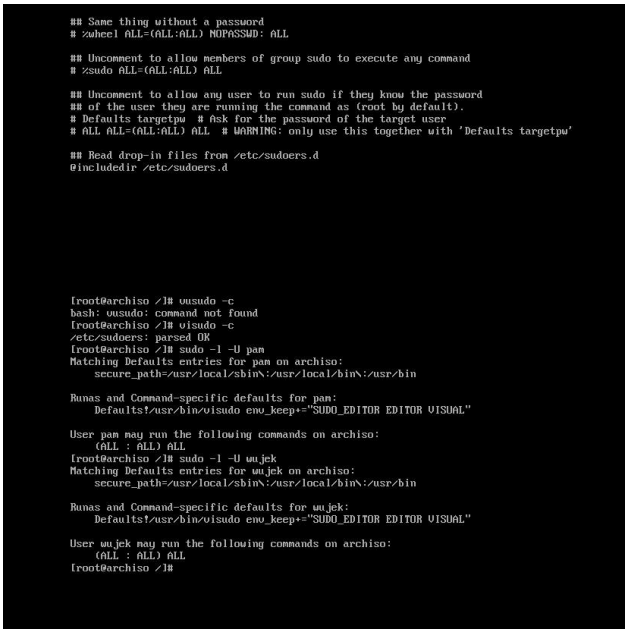
```
%wheel ALL=(ALL:ALL) ALL
```

Save and exit. Check the configuration:

```
visudo -c          # Verify valid sudoers syntax
sudo -l -U pam     # Show pam's allowed commands
sudo -l -U wujek   # Show wujek's allowed commands
```



Allowing `wheel` members to use sudo with a password.



Confirming sudo access for both accounts.

Step 18 — Enable SSH

```
pacman -Q openssh           # Check that OpenSSH is installed systemctl
enable sshd       # Start SSH automatically on future boots systemctl
is-enabled sshd   # Verify that it is enabled
```

`sshd` is the server component that accepts SSH connections. To connect from another computer later, verify the VM's IP address and network/firewall configuration.

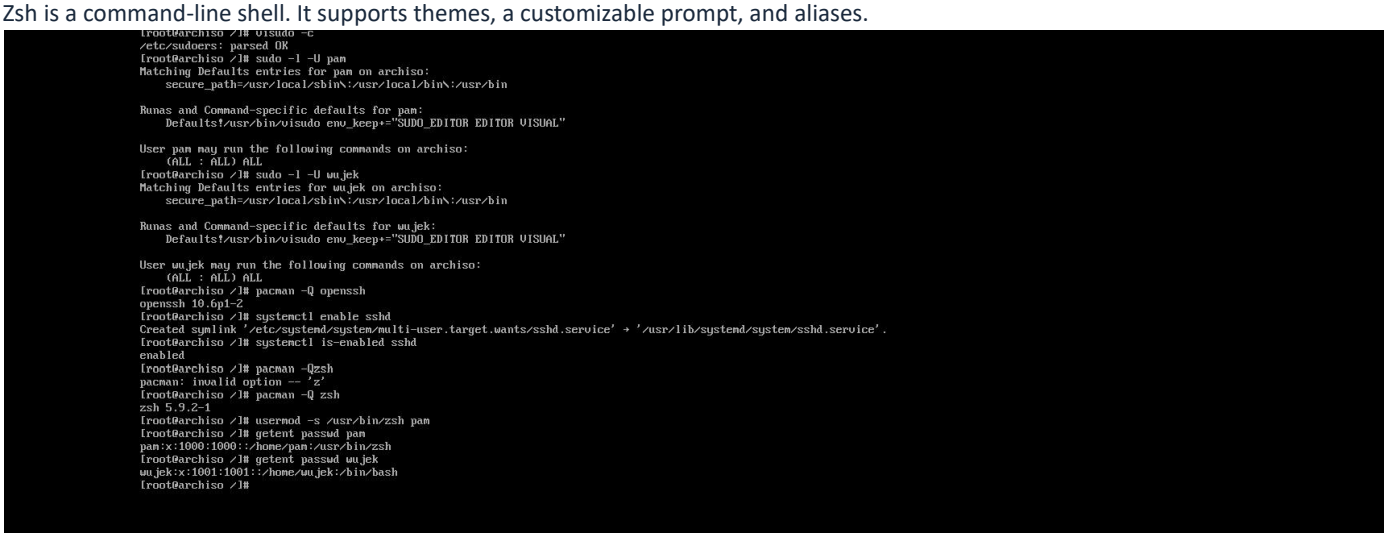


Enabling the SSH service for future boots.

Step 19 — Set Zsh as the main user's shell

```
pacman -Q zsh                  # Confirm Zsh is installed
usermod -s /usr/bin/zsh pam    # Make Zsh pam's login shell
getent passwd pam              # Verify pam ends in /usr/bin/zsh
getent passwd wujek            # Verify wujek still uses Bash
```

Zsh is a command-line shell. It supports themes, a customizable prompt, and aliases.



My main user uses Zsh and the second account uses Bash.

Step 20 — Reboot into the installed system

```
exit            # Leave the chroot environment
umount -R /mnt  # Unmount the installed disk from the live system
reboot          # Restart the VM
```

In VMware, disconnect the ISO so the VM boots from the virtual hard disk instead of returning to the installer.

Sign in as `pam` with the password you created.

---

5. Install and Use the Xfce Desktop

These commands run **after the first boot**, signed in as `pam` with working sudo permissions.

Step 21 — Update system packages

```
sudo pacman -Syu  # Refresh package databases and install system updates
```

Confirm any proposed updates after reading the package list.

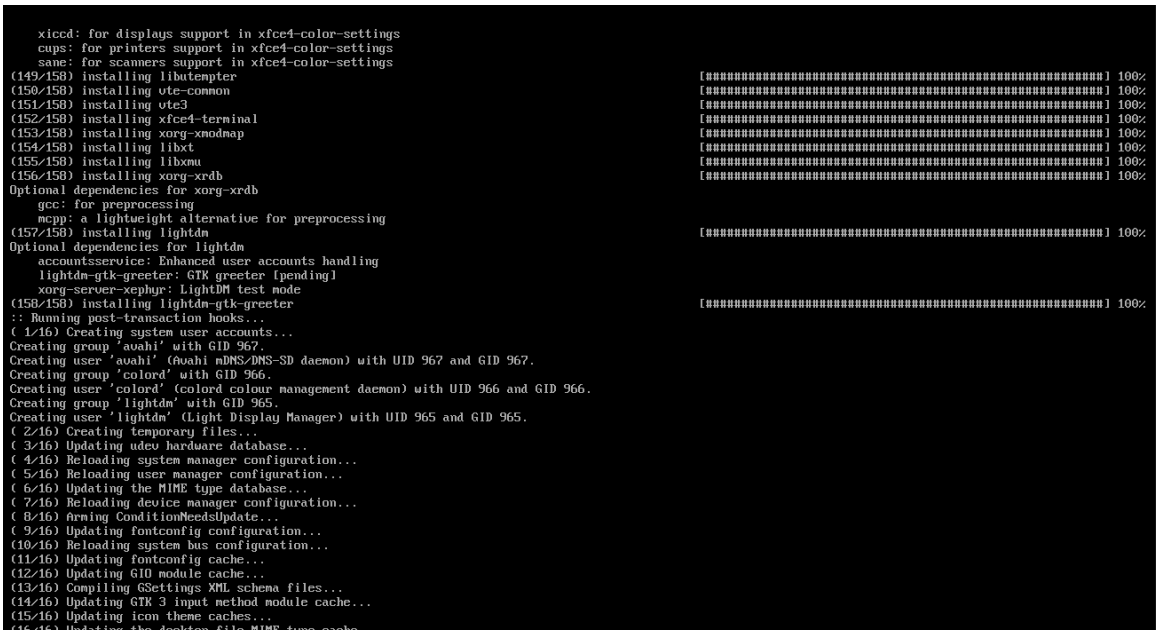
Step 22 — Install Xfce and the LightDM login manager

Xfce provides the desktop panel, window manager, file manager, terminal, and settings. LightDM provides a graphical login screen.

```
sudo pacman -S --
        needed xorg-server xfce4-session xfce4-panel xfwm4 xfdesktop xfce4-settings xfce4-terminal thunar
        lightdm lightdm-gtk-greeter
```

After the packages finish, confirm the Xfce session and configure the graphical startup:

```
ls /usr/share/xsessions/              # Check that xfce.desktop exists
sudo systemctl enable lightdm.service  # Enable LightDM at startup
systemctl is-enabled lightdm.service  # Verify: enabled
sudo systemctl set-default graphical.target  # Boot into the GUI by default
systemctl get-default                 # Verify: graphical.target
sudo reboot                           # Restart into graphical login
```



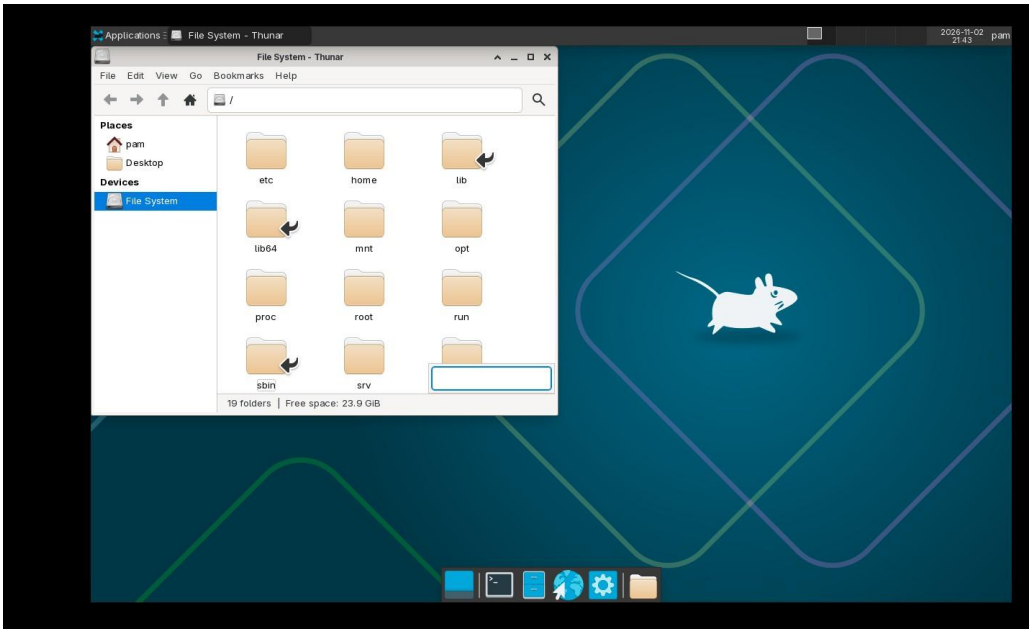
Xfce and LightDM packages installed successfully.



The Xfce session exists and LightDM is enabled.

Step 23 — Sign into the desktop

On the LightDM login screen, select **Xfce Session** if prompted, enter the username `pam`, and sign in. You should see an Xfce panel, desktop background, and the **Thunar** file manager.



Arch Linux booted into Xfce and ready to use.

---

6. Customize Zsh and Install Firefox

Step 24 — Add terminal colors and useful aliases

Open **Applications → Terminal** and edit the Zsh configuration file:

```
nano ~/.zshrc  # Customize the active user's shell
```

Paste the following configuration:

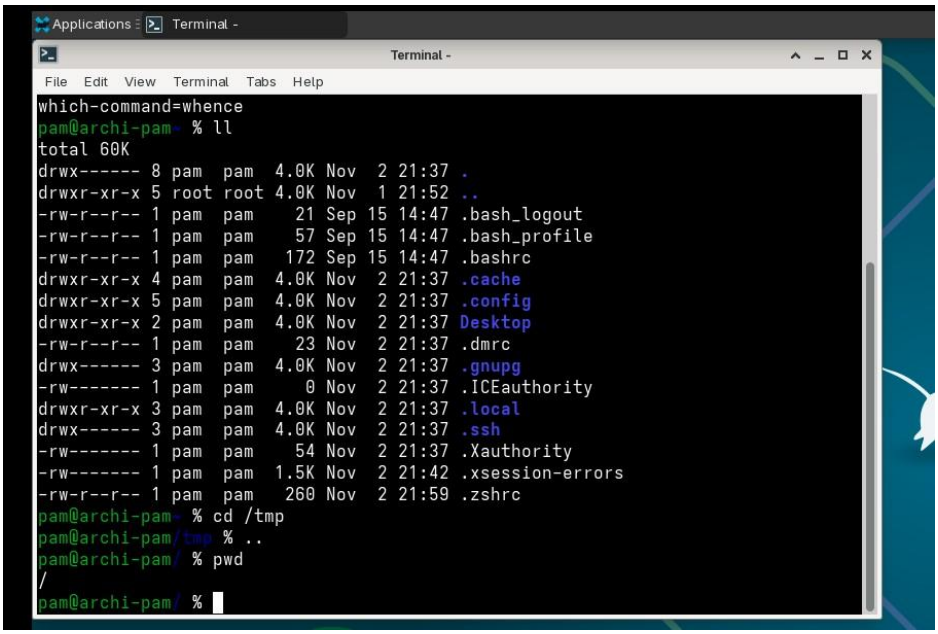
```
# Colorful output for common commands
alias ls='ls --color=auto'
alias ll='ls -lah --color=auto'
alias grep='grep --color=auto'

# Useful shortcuts
alias update='sudo pacman -Syu'
alias cls='clear'
alias ..='cd ..'

# Green username and blue current directory
autoload -Uz colors
colors
PROMPT='%F{green}%n@%m%f:%F{blue}%~%f %# '
```

Save with **Ctrl + O → Enter → Ctrl + X**, then reload the settings and test the aliases:

```
source ~/.zshrc  # Load the updated configuration now
alias            # List defined shortcuts
ll               # Show a detailed colored file listing
cd /tmp          # Change to /tmp
..               # Go up one directory
pwd              # Verify the current directory is /
```

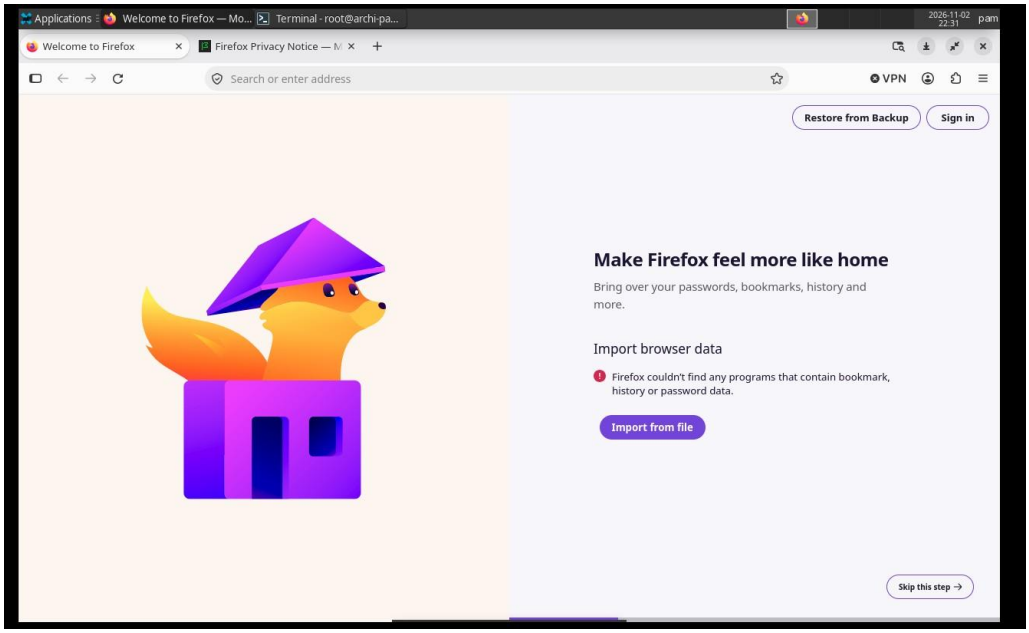


Green and blue Zsh prompt, color-coded file listing, and working directory alias.

Step 25 — Install the Firefox browser

```
sudo pacman -S firefox  # Install Firefox
pacman -Q firefox       # Verify the installed version
firefox                 # Launch the browser
```

If Pacman asks you to select packages that provid`jack` or `ttf-font`, my installation used`pipewire-jack` and `noto-fonts`. Once Firefox opens, browse toarchlinux.org to confirm internet access in the desktop.



Firefox successfully opened in Xfce.

---

7. Build a Package from the AUR

The **Arch User Repository (AUR)** provides community-maintained build recipes. My example is **Yay**, a command-line AUR helper written in Go. For safety, review a package's `PKGBUILD` before building it. Build AUR pacl egular user, not as `root`.

Step 26 — Install build prerequisites

```
sudo pacman -S --needed base-devel git   # Install compilers, make tools, and Git
pacman -Q base-devel git                 # Confirm they are available
```

Step 27 — Clone the Yay repository

```
cd ~                                     # Go to the home directory mkdir -p
aur-builds                          # Create a folder for AUR projects cd aur-
builds                                # Enter the new folder git clone
https://aur.archlinux.org/yay.git # Download Yay's build recipe cd yay
# Enter the repository
ls -la                                   # Look for PKGBUILD and .SRCINFO
```

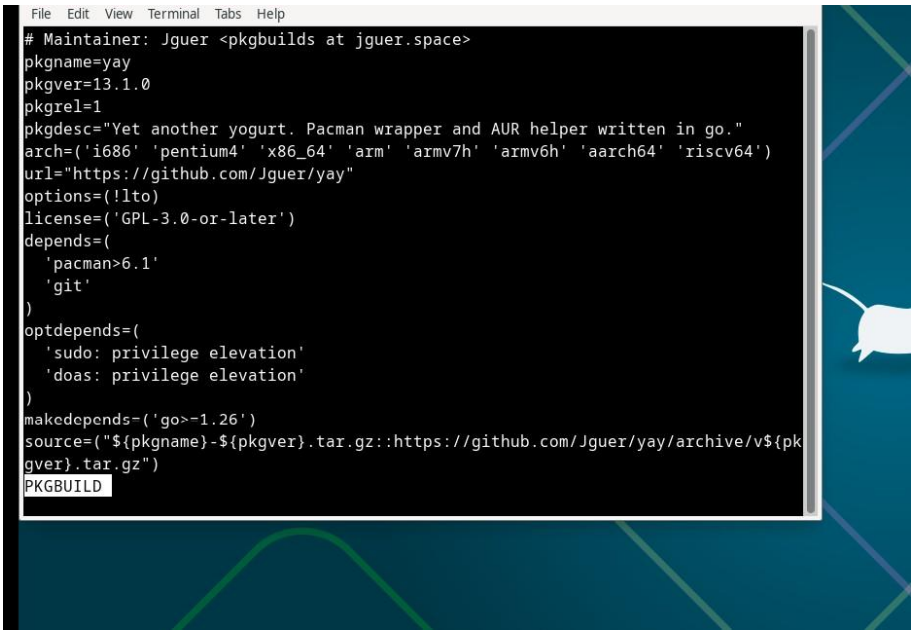
```
pam@archi-pam ~/aur-builds % git clone https://aur.archlinux.org/yay.git
Cloning into 'yay'...
remote: Enumerating objects: 573, done.
remote: Counting objects: 100% (573/573), done.
remote: Compressing objects: 100% (390/390), done.
remote: Total 573 (delta 181), reused 555 (delta 180), pack-reused 0 (from
Receiving objects: 100% (573/573), 112.28 KiB | 1.08 MiB/s, done.
Resolving deltas: 100% (181/181), done.
pam@archi-pam ~/aur-builds % cd yay
pam@archi-pam ~/aur-builds/yay % ls -la
total 24
drwxr-xr-x 3 pam pam 4096 Nov  2 22:40 .
drwxr-xr-x 3 pam pam 4096 Nov  2 22:40 ..
drwxr-xr-x 7 pam pam 4096 Nov  2 22:40 .git
-rw-r--r-- 1 pam pam   46 Nov  2 22:40 .gitignore
-rw-r--r-- 1 pam pam 1035 Nov  2 22:40 PKGBUILD
-rw-r--r-- 1 pam pam  634 Nov  2 22:40 .SRCINFO
```

The Yay repository includes `PKGBUILD` and `.SRCINFO`.

Step 28 — Read the build recipe

```
sudo pacman -S less  # Install a terminal file viewer if it is not available
less PKGBUILD        # Read the package metadata and build instructions
```

Use the arrow keys to scroll, then press `q` to close `less`. In the file, pay attention to `source=`, `depends=`, `makedepends=`, `build()`, and `package()`.

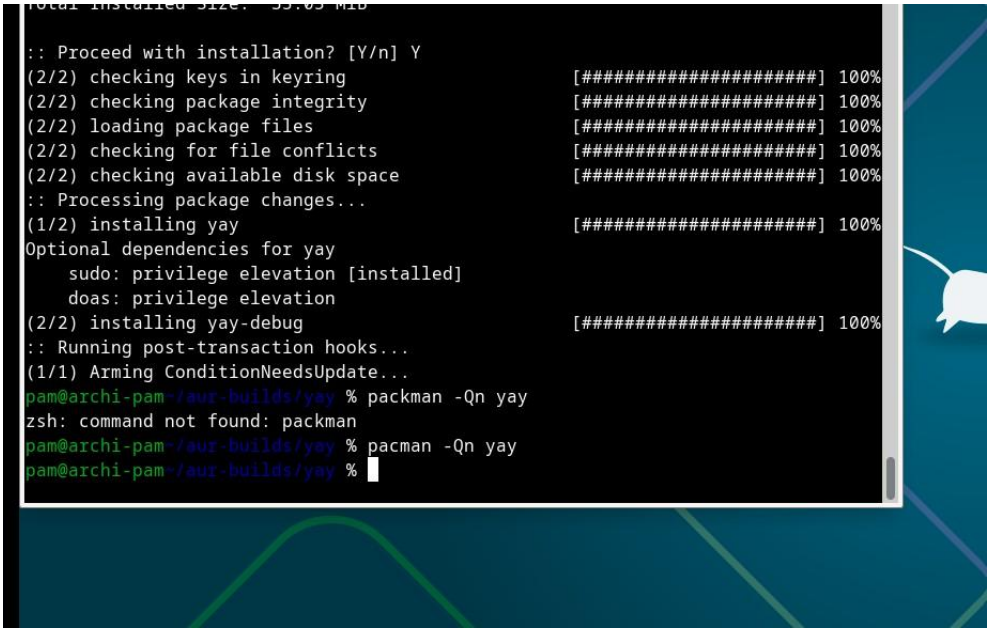


Inspecting the AUR build instructions before installing the package.

Step 29 — Build and install Yay

```
whoami       # Verify that the current user is pam (not root)
makepkg -si  # Download dependencies, build Yay, and install its package
```

`makepkg` prompts for administrative authorization when needed, s**do not prefix** `makepkg -si` **with** `sudo`.



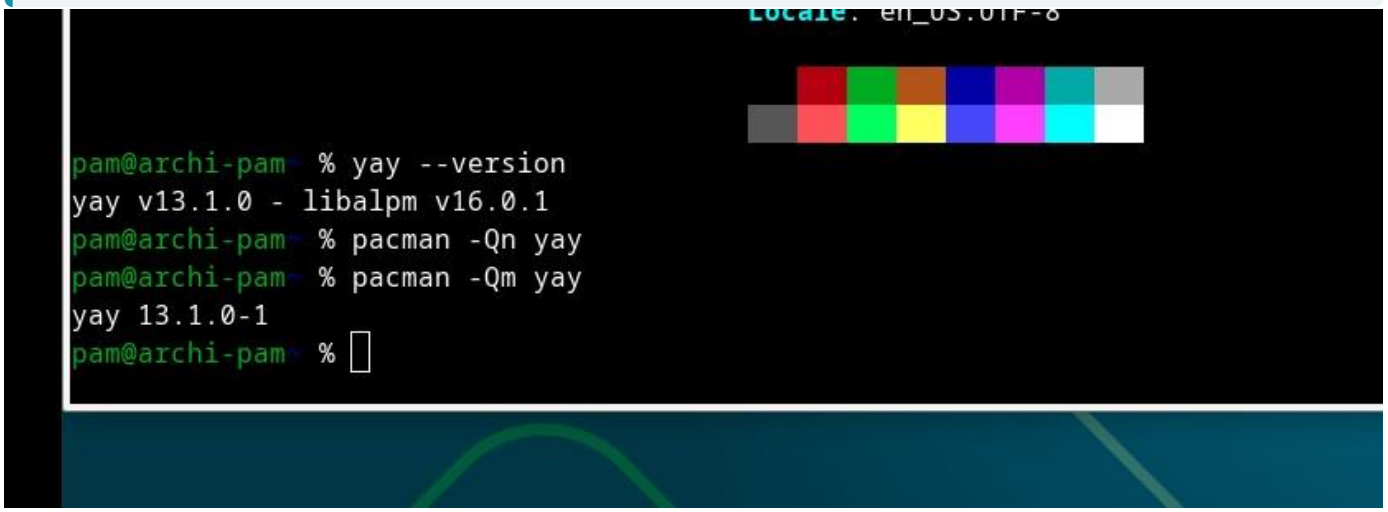
Yay installed as an Arch package.

Step 30 — Verify Yay

```
yay --version  # Print the Yay version
pacman -Qm yay # Show Yay as a foreign/AUR-installed package
```

My VM showed:

```
yay v13.1.0 - libalpm v16.0.1
yay 13.1.0-1
```



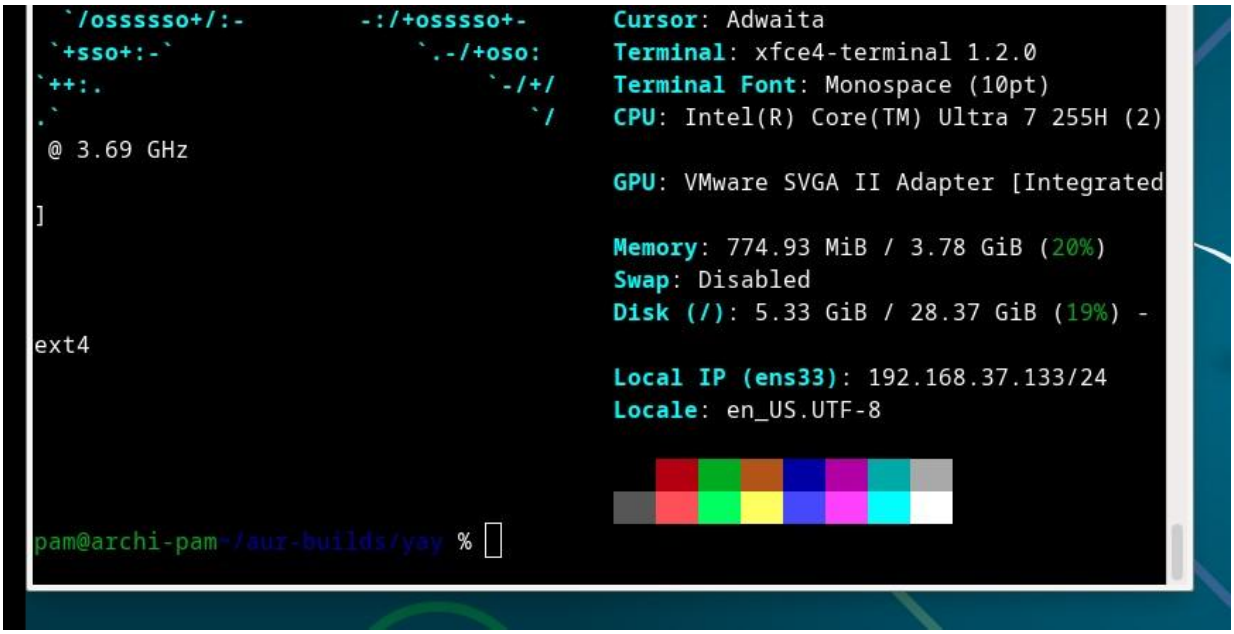
Verifying the Yay executable and the installed package.

8. Verify the Finished VM

Step 31 — Display the final Arch system details

Install Fastfetch if it is not present, then display the system overview:

```
sudo pacman -S fastfetch # Install a terminal-based system information tool
fastfetch                # Show OS, kernel, RAM, disk, and desktop details
whoami                   # Confirm active user
cat /etc/os-release      # Confirm Arch Linux
```



Fastfetch showing the running Arch Linux VM and virtual hardware.

Final setup checklist

Feature	Verification command or action

Arch installed	`cat /etc/os-release`
Correct user	`whoami` → `pam`
Two users created	`id pam` and `id wujek`
Sudo configured	`sudo -l`
SSH enabled	`systemctl is-enabled sshd`
Zsh shell	`echo $SHELL`
Xfce desktop	Log in to Xfce through LightDM
GUI boots by default	`systemctl get-default` → `graphical.target`
Terminal shortcuts	`ll`, `alias`, `..`
Browser	Launch Firefox and open a website
AUR package	`yay --version` and `pacman -Qm yay`
System overview	`fastfetch`

9. References

- Official Arch Installation Guide
- Arch Wiki — GRUB
- Arch Wiki — Xfce
- Arch Wiki — NetworkManager
- Arch Wiki — Arch User Repository
- GitHub Pages documentation

This beginner guide is based on the Arch Linux virtual machine I created and the screenshots captured during installation. For different hardware, check the official Arch Wiki and adjust device names, time zone, and VM settings accordingly.